CS4031 – Compiler Construction
Course Project Proposal
Spring 2026

Project Title: QuestScript - A CYOA Mini-Compiler

Theme: Choose Your Own Adventure

Team Members:

Student Name	Student ID
1. Syed Waleed Hussain	23K-0885
2. Huzaifa Altaf	23K-0627
3. Sofia Ayaz	23K-0807

---

1. Language Concept

"QuestScript" is a simple domain-specific language made for creating text-based interactive story games. Its main purpose is to let anyone easily write stories, define rooms, and add choices for the player without writing complex code. The compiler will read this script and run it as a playable game directly in the terminal.

2. Key Features

- Feature 1: "Scene" blocks to write the story text and define game events.
- Feature 2: A simple choice system (choice ->) that lets the player jump from one scene to another.
- Feature 3: Basic variables (true/false) to keep track of items or events (e.g., var has_key = false).
- Feature 4: Conditional statements (if/else) to restrict player choices based on the items they have.
- Feature 5: A clear game-over command (end) to finish the story.

3. Example Program

```
// Simple escape room logic in QuestScript
var has_key = false;

scene start {
    print "You wake up in a dark room.";
    choice "Search the floor" -> find_key;
    choice "Try the door" -> check_door;
}

scene find_key {
    set has_key = true;
    print "You found a rusty key!";
    goto start;
}
scene check_door {
    if has_key == true {
        print "Door unlocked. You escaped!";
        end;
    } else {
        print "The door is locked.";
        goto start;
    }
}
```

4. Target Output

- **Direct execution and display of results**
- **Other: Custom Python Interpreter / REPL**

Explanation: Our compiler will not generate assembly or object files. Instead, it will read the .adv source code and directly run the interactive story game in the terminal using a custom Python Virtual Machine.

5. Expected Challenges

- **Challenge 1:** Checking if a target scene actually exists when a player makes a choice to jump to it (this is known as Semantic Analysis and scope checking).
- **Challenge 2:** Designing the grammar rules so that they are simple enough to avoid parsing errors, especially when dealing with nested if/else conditions.
- **Challenge 3:** Building a stable, continuous loop (the Virtual Machine/REPL) that safely takes player input and updates the game variables without crashing.